Schiavinato Sharing: Manual and Computational Threshold Secret Sharing for BIP39 Mnemonics over GF(2053)

Renato Schiavinato Lopez*
GRIFORTIS
https://grifortis.com

April 10, 2026
v0.5.0

Abstract

Crypto-asset self-custody remains fragile because a single BIP39 mnemonic is both a single point of failure and a source of long-term tooling dependence. We present Schiavinato Sharing, a k-of-n threshold secret sharing scheme for BIP39 mnemonics over GF(2053), the smallest prime field that supports direct arithmetic on BIP39 word indices.

The construction admits both computational and fully manual operation: shares can be generated and recovered either by software or by hand using modular arithmetic and precomputed Lagrange coefficients. In author-measured benchmarks, manual recovery required about 30 minutes for 12-word 2-of-3 and about 60 minutes for 24-word 3-of-5 configurations.

Each arithmetic share carries a linear checksum architecture (the Arithmetic Lock) with minimum detection distance $d = 4$, so all 1-, 2-, and 3-cell errors are detected with certainty. Under a uniform random error model, the undetected-error probability ranges from $\approx 3.2 \times 10^{-27}$ for 12-word mnemonics to $\approx 1.8 \times 10^{-40}$ for 24-word mnemonics.

To address share substitution, the protocol combines pre-recovery manifest checks with post-recovery verification via a wallet-bound Recovery Verification Address and a computational Identity Lock. The construction also supports recursive composition up to four layers. Information-theoretic confidentiality for $t < k$ applies to the arithmetic share layer by standard Shamir/LSSS theory; optional verification metadata provides substitution detection but lies outside that unconditional-secrecy claim. Reference implementations and test vectors are available at https://github.com/GRIFORTIS/schiavinato-sharing.

1 Introduction

Self-custody of crypto assets depends on a single secret—a BIP39 mnemonic [3]—serving as the master seed for hierarchical deterministic wallets [2]. If this secret is lost, assets become unrecoverable; if exposed, they can be stolen. Traditional backup strategies concentrate risk and fail to provide resilience against environmental catastrophe or multi-decade technological obsolescence [9].

1.1 The Sovereignty Gap

True self-custody requires *sovereignty*: control without permission from, or dependency on, any third party. In practice, recovery methods can centralize trust in specific hardware,

*__Experimental, unaudited software; do not use for real funds.__ Reference implementations: https://github.com/GRIFORTIS/schiavinato-sharing. Responsible disclosure: security@grifortis.com.

proprietary software, and vendor-maintained formats. The ecosystem exhibits high fragmentation: proprietary formats from early hardware wallet generations, threshold schemes with incomplete adoption,[1] and discontinued software demonstrate that tools available at backup time may be unavailable at recovery time. When a user depends on a computational tool to reconstruct their secret, they have traded physical sovereignty for digital convenience.

1.2 Existing Approaches

Current threshold-based solutions often require trade-offs between computational efficiency and sovereign auditability.

Manual-capable schemes. Codex32 [7] (BIP-93) uses BCH error-correction codes and paper computation wheels (volvelles), providing error correction but requiring mastery of complex lookup procedures and non-BIP39 encoding. SeedXOR [13] offers XOR-based splitting but is inherently n-of-n and provides weak error detection.

Table 1: Manual-capable schemes (illustrative 24-word, 3-of-5 comparison)

Feature	**Schiavinato**	**Codex32**	**SeedXOR**
Threshold	Full k-of-n	Full k-of-n	n-of-n only
Manual complexity	Integer mod 2053	Volvelles/tables	XOR
Error handling	Detection (d=4)	Correction (BCH)	Weak/None
Composition	Recursive (4 layers)	Flat	Flat
Output format	Standard BIP39	Custom bech32	Standard BIP39
Illustrative recovery time	30–60 min[a]	4–8 hours[b]	1–2 hours[b]

a Author-measured, non-controlled timings (Section 7).
b Author estimates from published workflow descriptions; included only as rough order-of-magnitude context.

Electronic-only schemes. SLIP39 [15] and SSKR [16] implement Shamir sharing over extension fields (GF(2^{10}) and GF(2^8)). While cryptographically sound, arithmetic in GF(2)-extensions such as GF(2^{10}), performed via polynomial reduction modulo irreducible polynomials, is manually intractable. Recovery therefore depends on specialized software. SLIP39 uses a key derivation function incompatible with BIP39: recovering a SLIP39 share set does not produce a BIP39-compatible wallet. SSKR preserves BIP39 round-trip compatibility by operating on raw entropy, but shares use the custom Bytewords/UR encoding.

Table 2: Electronic threshold schemes (illustrative operational comparison)

Feature	**Schiavinato**	**SLIP39**	**SSKR**
Field	Prime GF(2053)	Extension GF(2^{10})	Extension GF(2^8)
Operation on	Word indices	Binary entropy	Binary entropy
BIP39 compatibility	Native I/O	Incompatible†	Round-trip‡
Share format	GF(2053) table + QR	Custom 20-word	Bytewords/UR
Manual operation	Complete lifecycle	No	No
Recovery-tool dependence	Low	High	Moderate

† SLIP39 uses a different key derivation function; the same entropy produces a different wallet than BIP39.
‡ SSKR operates on raw BIP39 entropy and preserves the key derivation path, enabling full round-trip. Shares use the Bytewords/UR format.

Distinct problem space. MPC wallets, multisignature schemes, and smart contract social recovery [6] address *operational security* for active asset management. Schiavinato Sharing addresses *cold storage and disaster recovery*—a complementary problem space.

[1] SLIP39 [15] support remains uneven across implementations.

1.3 Contributions

To the author's knowledge, no existing published scheme simultaneously offers BIP39 native input/output, flexible k-of-n thresholds, manual and computational execution as complementary modes, and auditable arithmetic. Schiavinato Sharing addresses this gap with the following contributions:

1. **Algebraic mapping:** Shamir Secret Sharing over GF(2053) with direct BIP39 word-index arithmetic, making the complete sharing and recovery lifecycle executable by hand with a calculator.
2. **Integrity architecture:** A linear checksum system with provable detection distance $d = 4$, enabling incremental error detection during manual ceremonies without electronic verification.
3. **Substitution detection:** Layered mechanisms—manifest cross-checks (pre-recovery) and cryptographic identity binding (post-recovery)—that detect share tampering across trust boundaries and over time.
4. **Recursive composition:** Support for up to four nesting layers, enabling complex multi-party custody structures within a single protocol.

1.4 Paper Organization

Section 2 reviews prerequisites. Section 3 specifies the construction. Section 4 formalizes the integrity architecture and proves its detection properties. Section 5 analyzes security. Section 6 provides a worked example. Section 7 describes implementations and preliminary evidence. Section 8 discusses limitations and open directions. Appendices contain payload encoding reference and pre-computed Lagrange coefficients.

2 Preliminaries

2.1 Shamir's Secret Sharing

Shamir's Secret Sharing (SSS) [14] splits a secret s into n shares such that any k suffice for reconstruction and any $t < k$ reveal zero information about s.

Definition 2.1.1 (Shamir Sharing over GF(p))**.** *Given secret $s \in \mathrm{GF}(p)$, threshold k, and n share indices $x_1, \ldots, x_n \in \mathrm{GF}(p) \setminus \{0\}$:*

1. *Sample $a_1, \ldots, a_{k-2} \xleftarrow{\$} \mathrm{GF}(p)$ and $a_{k-1} \xleftarrow{\$} \mathrm{GF}(p) \setminus \{0\}$.*
2. *Define $f(x) = s + a_1 x + \cdots + a_{k-1} x^{k-1} \pmod p$.*
3. *Output shares $(x_i, f(x_i))$ for $i = 1, \ldots, n$.*

Recovery uses Lagrange interpolation: $s = f(0) = \sum_{j=1}^{k} \gamma_j \cdot y_j \pmod p$ where $\gamma_j = \prod_{i \neq j} \frac{x_i}{x_i - x_j} \pmod p$.

Security. Information-theoretic (unconditional) secrecy: for any two candidate secrets s_0, s_1, the distribution of any $t < k$ shares is identical. This holds regardless of adversary computational resources.

2.2 Linear Secret Sharing Schemes

A Linear Secret Sharing Scheme (LSSS) [1] is one where share values are linear functions of underlying secrets. By standard theory [8], public linear relationships among shared secrets do not compromise information-theoretic security: authorized sets uniquely determine all secrets; unauthorized sets reveal zero information. This holds for arbitrary distributions on the secret space, including non-uniform distributions induced by external constraints [5].

2.3 BIP39 Mnemonics

A BIP39 mnemonic [3] encodes cryptographic entropy as a sequence of $\ell \in \{12, 15, 18, 21, 24\}$ words from a standard 2048-word list. Each word maps to an 11-bit index. For 24 words, the mnemonic encodes 256 bits of entropy plus an 8-bit SHA-256 checksum, yielding 2^{256} valid mnemonics out of $2048^{24} \approx 2^{264}$ possible sequences. An optional passphrase ("25th word") participates in seed derivation but is external to the mnemonic encoding.

3 Construction

3.1 Field Selection: GF(2053)

The field $\mathrm{GF}(p)$ with $p = 2053$, the smallest prime exceeding 2048, provides:

1. **Direct word-index mapping:** every BIP39 word index $w \in \{1, \ldots, 2048\}$ is representable in GF(2053) without encoding overhead, using 1-based indexing (abandon = 1, zoo = 2048).
2. **Manual arithmetic:** prime field operations reduce to integer arithmetic with modular reduction, executable with a basic calculator.
3. **Standard security:** GF(2053) is a standard finite field with no structure weakening Shamir's guarantees.

Since $|\,\mathrm{GF}(2053)| > 2048$, approximately 0.24% of share values may fall outside the BIP39 range $\{1, \ldots, 2048\}$. On share sheets, in-range values are displayed alongside their BIP39 word for redundancy and better readability; out-of-range values in {0, 2049, 2050, 2051, 2052} use numeric-only placeholders. This does not affect security or recovery correctness.

3.2 Sharing Algorithm

Parameters and notation.

- **Threshold:** (k, n) with $2 \le k \le n \le 2052$ (see Section 3.4 for $k = 1$); share indices must be distinct nonzero elements of GF(2053)
- **Mnemonic:** $(w_1, \ldots, w_\ell) \in \{1, \ldots, 2048\}^\ell$ with $\ell \in \{12, 15, 18, 21, 24\}$
- **Row count:** $r = \ell/3$ (the mnemonic is arranged as r rows $\times$ 3 columns)
- **Share indices:** $x \in \{1, \ldots, n\} \subset \mathrm{GF}(2053) \setminus \{0\}$
- **Row tags:** $\tau_j^R = j$ for $j = 1, \ldots, r$
- **Column tags:** $\tau_c^C \in \{10, 20, 30\}$ for $c = 1, 2, 3$
- **Row total:** $T_R = \sum_{j=1}^{r} j = r(r+1)/2$
- **Column total:** $T_C = 10 + 20 + 30 = 60$ (invariant across configurations)

Row and column tags serve as domain separators ensuring positional binding. Column tags are chosen from $\{10, 20, 30\}$ to be disjoint from row indices $\{1, \ldots, r\}$ for all supported mnemonic lengths ($r \le 8$), enabling unambiguous error attribution during manual verification.

Step 1: Word polynomial generation. For each word w_i, $i = 1, \ldots, \ell$:

- Sample coefficients: $a_1, \ldots, a_{k-2} \xleftarrow{\$} \{0, \ldots, 2052\}$; $a_{k-1} \xleftarrow{\$} \{1, \ldots, 2052\}$.
- Define $f_{w_i}(x) = w_i + a_1 x + \cdots + a_{k-1} x^{k-1} \pmod{2053}$.
- Evaluate: $y_{w_i}^{(x)} = f_{w_i}(x)$ for each share index x.

Step 2: Row checksum computation and incremental validation. For each row $j = 1, \ldots, r$:

$$f_{R_j}(x) = \big(f_{w_{3j-2}}(x) + f_{w_{3j-1}}(x) + f_{w_{3j}}(x)\big) + \tau_j^R \pmod{2053} \tag{1}$$

where $\tau_j^R = j$ is added to the constant term only. No additional randomness is required. At any share index x:

$$y_{R_j}^{(x)} = \left(y_{w_{3j-2}}^{(x)} + y_{w_{3j-1}}^{(x)} + y_{w_{3j}}^{(x)} + j\right) \bmod 2053 \tag{2}$$

This row-level structure enables *incremental validation* during share construction: after computing every three word shares and their row checksum, each share's row values can be immediately verified via Equation (2), catching arithmetic or transcription errors before proceeding to the next row.

Step 3: Column checksum computation. For each column $c = 1, 2, 3$:

$$f_{C_c}(x) = \sum_{\substack{i:\text{word } i \\ \text{is in column } c}} f_{w_i}(x) + \tau_c^C \pmod{2053} \tag{3}$$

At any share index x:

$$y_{C_c}^{(x)} = \left(\sum_{i \in \text{col } c} y_{w_i}^{(x)} + \tau_c^C\right) \bmod 2053 \tag{4}$$

Step 4: Global Integrity Check (GIC). Define the base GIC polynomial by summing all word polynomials and adding the row and column totals to the constant term (enabling addition-only validation paths):

$$f_{\text{GIC}}(x) = \sum_{i=1}^{\ell} f_{w_i}(x) + T_R + T_C \pmod{2053} \tag{5}$$

The *share-bound* GIC on each share adds the share number for index binding:

$$\text{GIC}(x) = \left(f_{\text{GIC}}(x) + x\right) \bmod 2053 \tag{6}$$

Three equivalent validation paths hold at any share index x:

$$\text{GIC}(x) \equiv \sum_{i=1}^{\ell} y_{w_i}^{(x)} + T_R + T_C + x \pmod{2053} \tag{7}$$

$$\equiv \sum_{j=1}^{r} y_{R_j}^{(x)} + T_C + x \pmod{2053} \tag{8}$$

$$\equiv \sum_{c=1}^{3} y_{C_c}^{(x)} + T_R + x \pmod{2053} \tag{9}$$

Step 5: Share assembly. Each share at index x contains $\ell + r + 3 + 1$ field elements:

$$S_x = \left(y_{w_1}^{(x)}, \ldots, y_{w_\ell}^{(x)},\ y_{R_1}^{(x)}, \ldots, y_{R_r}^{(x)},\ y_{C_1}^{(x)}, y_{C_2}^{(x)}, y_{C_3}^{(x)},\ \text{GIC}(x)\right)$$

For a 24-word mnemonic: $24 + 8 + 3 + 1 = 36$ values per share.

BIP39 passphrase independence. The BIP39 passphrase is external to the sharing arithmetic and is never encoded in share payloads. Users relying on a passphrase must back it up separately.

3.3 Recovery Algorithm

Input. k shares $S_{x_1}, \ldots, S_{x_k}$.

Step 1: Per-share validation. For each share at index x, verify the share-bound GIC using any path from Equations (7)–(9). The $+x$ term binds the GIC to the share number: if x is mislabeled, validation fails.

Step 2: Lagrange coefficient computation and sanity check. Compute $\gamma_j = \prod_{i \neq j} x_i/(x_i - x_j) \pmod{2053}$ for $j = 1, \ldots, k$. Verify:

$$\sum_{j=1}^{k} \gamma_j \cdot x_j \equiv 0 \pmod{2053} \tag{10}$$

This holds because Lagrange interpolation of $g(x) = x$ at $x = 0$ yields $g(0) = 0$. Failure indicates coefficient error.

Lagrange coefficients are determined entirely by share indices and contain zero secret information. They may be pre-computed for common schemes and published (Appendix B), or computed on any device—even untrusted—without leaking secrets.

Step 3: Row-by-row interpolation with incremental validation. For each row $j = 1, \ldots, r$, interpolate the three word values and the row checksum:

$$\hat{s} = \sum_{m=1}^{k} \gamma_m \cdot y_s^{(x_m)} \pmod{2053}$$

for $s \in \{w_{3j-2},\, w_{3j-1},\, w_{3j},\, R_j\}$. Immediately verify:

$$(\hat{w}_{3j-2} + \hat{w}_{3j-1} + \hat{w}_{3j} + j) \bmod 2053 \stackrel{?}{=} \hat{R}_j$$

A mismatch localizes the error to the current row, allowing correction before proceeding. This incremental structure is the primary error-catching mechanism during manual recovery.

Step 4: Global validation. After all rows pass, interpolate the three column checksums $\hat{C}_1, \hat{C}_2, \hat{C}_3$ and verify each via Equation (4). Verify the recovered GIC via Equations (7)–(9).

The $+x$ terms in the share-bound GIC cancel automatically during interpolation by Equation (10). The row tags τ_j^R and column tags τ_c^C do *not* cancel, as they are embedded in the constant terms of the checksum polynomials.

Optional GIC cross-check. If share-bound GIC values are available from all k shares, they may be independently interpolated at $x = 0$. The $+x$ terms cancel, yielding $f_{\text{GIC}}(0) = \sum_{i=1}^{\ell} w_i + T_R + T_C$. Comparing this against the same sum computed from recovered words provides an independent validation path that does not depend on row or column checksums.

Step 5: Output. Convert recovered word indices $(\hat{w}_1, \ldots, \hat{w}_\ell)$ to BIP39 words using the 1-indexed wordlist.

3.4 Recursive Composition (Nesting)

The construction supports recursive composition: a completed share from layer L becomes the protected object of layer $L+1$. Each layer is standard Shamir over GF(2053) with independently computed row checksums, column checksums, and GIC.

Intermediate-layer semantics. At layer $L > 0$, the "word" values being shared are GF(2053) elements from the parent share, not BIP39 word indices. Values $\{0, 2049, 2050, 2051, 2052\}$ are valid inputs. BIP39 validity is checked *only* at the final layer-0 recovery. Identity binding (Section 4.2.3) applies only at the outermost layer.

Degenerate case: $k = 1$. When $k = 1$, the polynomial is constant ($f(x) = a_0$) and all shares are identical copies of the parent value. This provides replication for access-control grouping (e.g., "any member of this group can represent the group") without threshold protection at that layer.

Depth limits. The sharing arithmetic itself imposes no nesting limit—any share can be recursively shared indefinitely. The practical depth constraint comes from the digital payload encoding, which allocates a fixed number of bits for per-layer threshold and index metadata (Section 4.2.1): Full Mode supports up to 4 active layers; Reduced Mode supports up to 2. Manual-only ceremonies without digital payloads are not subject to this limit.

4 Integrity Architecture

4.1 Arithmetic Lock: Detection Properties

The share table for an ℓ-word mnemonic contains $N = \ell + r + 3 + 1$ cells, where $r = \ell/3$. The $r + 4$ check equations (r row, 3 column, 1 GIC) define a linear code over GF(2053).

Definition 4.1.1 (Check equations)**.** *An error vector*

$$\mathbf{e} = (e_{w_1}, \ldots, e_{w_\ell}, e_{R_1}, \ldots, e_{R_r}, e_{C_1}, e_{C_2}, e_{C_3}, e_{GIC})$$

is undetectable *if it satisfies all $r + 4$ check equations:*

$$e_{w_{3j-2}} + e_{w_{3j-1}} + e_{w_{3j}} = e_{R_j} \qquad \textit{for } j = 1, \ldots, r \tag{11}$$

$$\sum_{i \in col\ c} e_{w_i} = e_{C_c} \qquad \textit{for } c = 1, 2, 3 \tag{12}$$

$$\sum_{i=1}^{\ell} e_{w_i} = e_{GIC} \tag{13}$$

Theorem 4.1.2 (Independence and rank)**.** *The $r + 4$ check equations are linearly independent over* $\mathrm{GF}(2053)^N$*. The check matrix has rank $r + 4$.*

Proof. Each row check equation (11) contains a unique variable e_{R_j} appearing in no other equation ($j = 1, \ldots, r$). Each column check equation (12) contains a unique variable e_{C_c} ($c = 1, 2, 3$). The GIC equation (13) contains a unique variable e_{GIC}. A system of $r + 4$ equations where each has at least one variable exclusive to it is linearly independent. □

Parity-check matrix. Let $H \in \mathrm{GF}(2053)^{(r+4)\times N}$ be the parity-check matrix whose rows are indexed by $R_1, \ldots, R_r, C_1, C_2, C_3, G$. Let $\mathbf{u}_{R_j}, \mathbf{u}_{C_c}, \mathbf{u}_G$ denote the corresponding standard basis vectors in $\mathrm{GF}(2053)^{r+4}$. The columns of H are

$$h_{j,c} = \mathbf{u}_{R_j} + \mathbf{u}_{C_c} + \mathbf{u}_G, \qquad h_{R_j} = -\mathbf{u}_{R_j}, \qquad h_{C_c} = -\mathbf{u}_{C_c}, \qquad h_G = -\mathbf{u}_G.$$

Here $h_{j,c}$ denotes the column corresponding to the word cell in row j and column c. By Definition 4.1.1, an error vector is undetectable if and only if it lies in $\ker H$.

Lemma 4.1.3 (Distance and dependent columns)**.** *For a linear code $C = \ker H$, the minimum Hamming weight of a nonzero codeword equals the minimum size of a linearly dependent set of columns of H.*

Proof. If $\mathbf{z} \in C$ is a nonzero codeword with support S, then $H\mathbf{z}^T = 0$ gives a nontrivial linear relation among the columns indexed by S, so those columns are linearly dependent. Conversely, if a set of columns indexed by S is linearly dependent, a nontrivial relation among them produces a nonzero vector $\mathbf{z} \in \ker H$ supported on S. Thus the smallest nonzero codeword weight equals the smallest dependent-column set size. □

Theorem 4.1.4 (Minimum detection distance)**.** *The minimum weight of a nonzero undetectable error vector is exactly $d = 4$. That is, all error patterns affecting 1, 2, or 3 cells are detected with certainty.*

Proof. By Lemma 4.1.3, it suffices to show that every set of 1, 2, or 3 columns of H is linearly independent, and that some set of 4 columns is linearly dependent.

One column. No column of H is zero, so no one-column dependence exists.

Two columns. If two nonzero columns are linearly dependent, one must be a nonzero scalar multiple of the other, hence they must have the same support. But all columns of H have distinct supports:

$$\operatorname{supp}(h_{j,c}) = \{R_j, C_c, G\}, \qquad \operatorname{supp}(h_{R_j}) = \{R_j\}, \qquad \operatorname{supp}(h_{C_c}) = \{C_c\}, \qquad \operatorname{supp}(h_G) = \{G\}.$$

Therefore every two-column set is linearly independent.

Three columns. Let m be the number of word columns in the set.

1. If $m = 0$, the columns are distinct signed basis vectors and are linearly independent.
2. If $m = 1$, say the set contains $h_{j,c}$, then any nontrivial dependence must cancel the coordinates G, R_j, and C_c. Among check columns, only h_G can cancel G, only h_{R_j} can cancel R_j, and only h_{C_c} can cancel C_c. Thus a dependence involving $h_{j,c}$ requires all three of those check columns in addition to $h_{j,c}$, for a total of 4 columns.
3. If $m = 2$, say the set contains h_{j_1,c_1} and h_{j_2,c_2}, then the G coordinate forces their coefficients to sum to zero, so their contribution is a nonzero scalar multiple of

   $$h_{j_1,c_1} - h_{j_2,c_2}.$$

   If the two word columns share neither row nor column, this difference has four nonzero row/column coordinates, which one check column cannot cancel. If they share a row, the difference equals $\mathbf{u}_{C_{c_1}} - \mathbf{u}_{C_{c_2}}$ and requires two column-check columns. If they share a column, it requires two row-check columns. Hence no three-column dependence exists with $m = 2$.
4. If $m = 3$, suppose there is a nontrivial dependence among three word columns. If any coefficient were zero, this would reduce to a one- or two-column dependence, already excluded. So all three coefficients are nonzero. View the three word columns as edges in a bipartite graph whose left vertices are rows and whose right vertices are columns. The row and column coordinates of the dependence imply that, at every incident vertex, the sum of coefficients on incident edges is zero. Therefore no incident vertex can have degree 1, since then the unique incident coefficient would be forced to zero. But every finite acyclic graph has a vertex of degree 1, so the selected three-edge subgraph must contain a cycle. A bipartite cycle has even length, hence at least 4, impossible with only 3 edges. Thus no three-word dependence exists.

Four columns (achievable). Dependent four-column sets exist, for example:

$$\begin{aligned} h_{j,c} + h_{R_j} + h_{C_c} + h_G &= 0, \\ h_{j_1,c_1} - h_{j_1,c_2} - h_{j_2,c_1} + h_{j_2,c_2} &= 0, \\ h_{j,c_1} - h_{j,c_2} + h_{C_{c_1}} - h_{C_{c_2}} &= 0, \\ h_{j_1,c} - h_{j_2,c} + h_{R_{j_1}} - h_{R_{j_2}} &= 0. \end{aligned}$$

These correspond respectively to the point, rectangle, row-pair, and column-pair undetectable error patterns.

Hence the smallest dependent column set has size 4, so the minimum weight of a nonzero undetectable error vector is exactly $d = 4$. □

Corollary 4.1.5 (Undetected-error probability)**.** *Under a uniform random error model over* $\mathrm{GF}(2053)^N$*, the probability that a nonzero error is undetectable is* $1/2053^{r+4}$ *for a mnemonic of* ℓ *words* ($r = \ell/3$)*. Concrete values:*

ℓ	r	*Checks* (r+4)	$P_{undetected}$
12	*4*	*8*	$1/2053^8 \approx 3.2 \times 10^{-27}$
15	*5*	*9*	$1/2053^9 \approx 1.5 \times 10^{-30}$
18	*6*	*10*	$1/2053^{10} \approx 7.5 \times 10^{-34}$
21	*7*	*11*	$1/2053^{11} \approx 3.7 \times 10^{-37}$
24	*8*	*12*	$1/2053^{12} \approx 1.8 \times 10^{-40}$

Proof. By Theorem 4.1.2, the null space of the check matrix has dimension $N - (r + 4)$. The fraction of nonzero vectors in the null space is $(2053^{N-(r+4)} - 1)/(2053^N - 1) \approx 1/2053^{r+4}$. □

Practical note. Under realistic (non-uniform) error models, the deterministic detection of all weight-≤ 3 errors (Theorem 4.1.4) is the primary assurance for manual use; the $1/2053^{r+4}$ bound applies to worst-case uniform random corruption.

Explicit limitation. The Arithmetic Lock detects random arithmetic and transcription errors. It provides zero detection of intentional substitution: an adversary with physical access to a share can choose arbitrary word values and compute consistent checksums trivially (Section 5.4).

4.2 Digital Envelope: Computational-Mode Integrity

Computational mode augments the Arithmetic Lock with two additional mechanisms, forming a layered integrity architecture. The reference byte-level encoding is specified in Appendix A.

4.2.1 Payload Profiles

Two profiles balance security, payload size, and manual transcribability:

Table 3: Full Mode vs. Reduced Mode

Property	**Full Mode**	**Reduced Mode**
Word-share range	$\{0, \ldots, 2052\}$ (12-bit)	$\{0, \ldots, 2047\}$ (11-bit)†
QR grid	37×37 (V5)	29×29 (V3)
Integrity model	Triple-Lock	Dual-Lock
Transport Hash	16 bytes (SHA-256)	Omitted‡
Blinded Identity	12 bytes ($\sim 2^{96}$)	8 bytes ($\sim 2^{64}$)
Session Batch ID	8 bytes	4 bytes
Nesting depth	Up to 4 layers	Up to 2 layers
Range-restriction bias	None	Small; conservatively bounded§

† Share-bound GIC remains 12-bit (GF(2053)). Word polynomials producing any share value > 2047 are rejected and regenerated.

‡ Transport integrity relies on QR error correction and GIC validation ($1 - 1/2053 \approx 99.95\%$). The 4 bytes are reallocated to the Blinded Identity.

§ Depends on mnemonic length ℓ and the number $m = n - t$ of unseen share positions. Section 5.3 gives the bound $\ell \cdot \log_2\big(2053/(2053 - 5m)\big)$. For 24-word examples, the common maximal-unauthorized cases remain below 0.26 bits; across all unauthorized sets of the currently emphasized topologies, the worst case is below 0.43 bits.

Reduced Mode is intended as an accessible alternative for short-term or budget-constrained use where manual QR transcription is required (a 3-of-5 scheme requires 2,640 fewer grid squares per session). Full Mode is recommended for long-term, high-value custody.

4.2.2 Transport Lock (Full Mode)

Each share's digital payload includes a 16-byte truncated SHA-256 [10] hash of the encoded header, session metadata, and share data. This provides 2^{-128} detection of random bit flips, scanning errors, or physical damage before mathematical processing.

4.2.3 Identity Lock

Master Key Identifier (MKI). The software derives the BIP32 master key from the mnemonic with empty passphrase, computes the compressed master public key (33 bytes), and takes its HASH160:

$$\text{MKI} = \text{RIPEMD-160}\big(\text{SHA-256}(\text{compressed_pubkey})\big) \quad (20 \text{ bytes})$$

This is the same intermediate value used to derive the standard 4-byte BIP32 fingerprint [2], but Schiavinato uses the full 20 bytes as the HMAC key.

Blinded Identity. Generated using HMAC-SHA256 [11]:

$$\text{BI} = \text{Trunc}_b\big(\text{HMAC-SHA256}(\text{key} = \text{MKI},\ \text{msg} = \text{SessionBatchID})\big)$$

where $b = 96$ bits (Full Mode) or $b = 64$ bits (Reduced Mode).

Why 20-byte MKI. If MKI were the standard 4-byte fingerprint, an adversary could enumerate all 2^{32} possible keys in seconds, find the one producing the target Blinded Identity, then generate a mnemonic with that fingerprint. The 20-byte MKI ensures the key space (2^{160}) is not the bottleneck; security is bounded by the Blinded Identity truncation length.

Verification. After recovery, the software re-derives MKI from the recovered mnemonic and recomputes BI. Mismatch triggers a strong warning (possible share mixing, corruption, or substitution). The MKI is internal only and never appears on shares.

4.2.4 Recovery Verification Address (RVA)

The Identity Lock is mnemonic-bound and requires software for verification. As a complementary wallet-bound check, the protocol allows recording a Recovery Verification Address: a user-supplied truncated verification address derived from the intended target wallet and written in the form "first 8 characters + last 8 characters" (e.g., `bc1qar0s...zzwf5mdq`).

Wallet context. The RVA is external metadata rather than part of the sharing arithmetic. It is derived from the recovered mnemonic together with the intended wallet context, which may include derivation settings or wallet profile and, if applicable, the BIP39 passphrase. The corresponding derivation hint must therefore be recorded separately.

Properties.

- **Substitution detection:** after recovery, recomputing the same target-wallet address and comparing it to the recorded RVA detects share substitution and wallet-context errors independently of any manifest.
- **Matching strength:** for bech32/bech32m addresses, the first 8 characters include 4 variable data characters (20 bits) and the last 8 characters include 2 data characters (10 bits) plus the 6-character checksum (30 bits), yielding $\sim 2^{60}$ matching power. This is the conservative benchmark among common Bitcoin address families; base58check truncations typically expose more visible entropy and therefore provide stronger matching but weaker privacy.

- **Privacy:** the truncated form reduces direct address disclosure and casual blockchain-look-up leakage relative to recording a full address. The RVA is human-readable metadata only and is not serialized in the digital payload.
- **Passphrase and derivation verification:** unlike the mnemonic-only Identity Lock, the RVA may validate the final wallet context, including passphrase and derivation settings, when those were part of the original wallet.

The RVA requires a one-time computational derivation during setup. In fully electronics-free ceremonies, this field may be omitted, accepting reduced post-recovery substitution detection.

4.3 Integrity Summary

Table 4: Integrity mechanisms by mode

Mechanism	Detects	Timing	Full	Reduced	Manual
Arithmetic Lock	Random errors (d=4)	On-the-fly	✓	✓	✓
Transport Lock	Media corruption	Pre-recovery	✓	—	—
Manifest GIC	Accidental mixing†	Pre-recovery	✓	✓	✓
Manifest Audit Hash	Substitution†	Pre-recovery	✓	✓	—
Identity Lock (BI)	Substitution ($\sim 2^{96}/2^{64}$)	Post-recovery	✓	✓	—
RVA	Substitution‡	Post-recovery	✓	✓	✓∗
BIP39 checksum	Invalid mnemonic	Post-recovery	✓	✓	✓

† Conditional on an uncompromised manifest.
‡ $\sim 2^{60}$ for bech32/bech32m truncation; typically stronger for base58check truncations.
∗ Requires a one-time computational step during the sharing ceremony to derive and record the RVA. Verification at recovery time requires any wallet software (offline, no blockchain access).

5 Security Analysis

5.1 Threat Model

Setup models. *Self-directed:* user generates own shares; dealer and user are identical. *Assisted:* a professional advisor provides guidance while the user maintains exclusive control of entropy and share handling.

Both models assume: (1) every device that handles secret-bearing material during setup or recovery—including the primary computation device and any printer, scanner, camera, or electronic calculator used in the workflow—is uncompromised and does not persist or exfiltrate sensitive data; (2) cryptographically secure random number generation; (3) accurate procedure execution; (4) shares are stored securely; and (5) if a manifest is used, it is treated as distinct sensitive metadata and stored separately, since manifest-dependent pre-recovery checks fail if the manifest is compromised. Manual sharing and recovery additionally require a private, surveillance-free environment.

Adversary. Standard Shamir adversary obtaining fewer than k shares. Mode-specific adversaries target side-channels, malicious dependencies or firmware, and supply-chain compromise of devices or peripherals that process secret-bearing data (computational mode), or physical observation, impression attacks, and waste recovery (manual mode).

Out of scope. Compromise of k or more shares, physical coercion, compromised devices or peripherals that observe, persist, or exfiltrate secret-bearing data, social engineering, standard Shamir attacks [14, 17], and underlying BIP39 ecosystem vulnerabilities.

Operational mitigations. The intended deployment model favors air-gapped or minimal-purpose devices, non-networked peripherals for secret-bearing output, offline verification, explicit memory cleanup, and workflows that minimize persistent secret-bearing artifacts. These mitigations apply to secret-bearing artifacts; manifest-only metadata is treated separately in Section 5.4.

5.2 Confidentiality

Proposition 5.2.1 (Information-theoretic confidentiality of the arithmetic share layer)**.** *For any $t < k$ and any two BIP39-valid mnemonics M_0, M_1, the distribution of any t arithmetic shares is identical, where an arithmetic share consists of the* GF(2053) *word-share values together with the derived checksum fields (row checksums, column checksums, and GIC).*

Proof sketch. The ℓ word indices are protected by independent degree-$(k-1)$ Shamir polynomials with uniformly random coefficients over GF(2053). By Shamir's theorem [14], the distribution of $t < k$ word-share values is identical for any constant term. The checksum fields (row checksums, column checksums, and GIC) are deterministic affine functions of the word shares with only public coefficients and public offsets (τ_j^R, τ_c^C, T_R, T_C, x), adding zero information to the adversary's view. By LSSS theory [1, 8], restricting the secret domain to BIP39-valid mnemonics does not change this: perfect secrecy is distribution-agnostic. □

Scope. Proposition 5.2.1 applies only to the unrestricted arithmetic share layer, including Full Mode and manual operation. Reduced Mode intentionally departs from exact perfect secrecy because rejection sampling conditions the word polynomials on the serialized range restriction; this deviation is analyzed separately in Section 5.3. Optional authenticity metadata such as the Blinded Identity and Recovery Verification Address strengthen substitution detection, but they are verification tags and are not covered by this unconditional-secrecy claim.

Proposition 5.2.2 (Entropy preservation)**.** *Effective keyspace remains $\approx 2^{256}$ for 24-word mnemonics. Sharing over* GF(2053) *introduces no compression.*

Justification. BIP39 entropy for $\ell = 24$ is 256 bits, determining 2^{256} valid mnemonics. Each word index $w_i \in \{1, \ldots, 2048\} \subset \mathrm{GF}(2053)$ is mapped injectively into the field without compression. The ℓ independent Shamir polynomials operate on these indices word by word; no entropy is combined, truncated, or hashed during sharing. The integrity fields are deterministic linear functions of the word shares and add no entropy of their own. □

Note on the coupled-constraint question. A previous version of this work [12] posed confidentiality under coupled BIP39/linear constraints as an open question. We observe that this follows directly from LSSS theory: Shamir's perfect secrecy is distribution-agnostic, and the linear integrity fields add no information beyond the t word share values already provided. We invite independent formal verification.

5.3 Reduced Mode Range-Restriction Bias

In Reduced Mode, word polynomials are rejected if any share evaluation exceeds 2047. This introduces a small bias because accepted polynomials are sampled from a restricted subset of the Shamir polynomial space. Rather than estimate this effect from a uniqueness or independence heuristic, we give a conservative bound that applies to any unauthorized set $t < k$.

Lemma 5.3.1 (Uniformity at one unseen position)**.** *Fix distinct points $0, x_1, \ldots, x_t, x^\star \in$* GF(2053) *with $t < k-1$, and fix values $s, y_1, \ldots, y_t \in$* GF(2053)*. Let $\mathcal{A}$ be the affine space of polynomials f of degree at most $k-1$ satisfying*

$$f(0) = s, \qquad f(x_i) = y_i \quad \text{for } i = 1, \ldots, t.$$

Then the evaluation map

$$\mathrm{ev}_{x^\star} : \mathcal{A} \to \mathrm{GF}(2053), \qquad f \mapsto f(x^\star)$$

is surjective, and every value in $\mathrm{GF}(2053)$ *has exactly* $|\mathcal{A}|/2053$ *preimages. Consequently, for any subset* $F \subset \mathrm{GF}(2053)$*, the fraction of polynomials in* $\mathcal{A}$ *with* $f(x^\star) \in F$ *is exactly* $|F|/2053$*.*

Proof. Define

$$g(X) = X \prod_{i=1}^{t} (X - x_i).$$

Since $t < k - 1$, we have $\deg g = t + 1 \leq k - 1$. Also, $g(0) = 0$ and $g(x_i) = 0$ for all $i = 1, \ldots, t$, while $g(x^\star) \neq 0$ because $x^\star$ is distinct from $0, x_1, \ldots, x_t$.

Let $f \in \mathcal{A}$ and let $z \in \mathrm{GF}(2053)$ be arbitrary. Setting

$$\lambda = \frac{z - f(x^\star)}{g(x^\star)}$$

gives $f + \lambda g \in \mathcal{A}$ and $(f + \lambda g)(x^\star) = z$. Thus $\mathrm{ev}_{x^\star}$ is surjective.

Its fibers are translates of $\ker(\mathrm{ev}_{x^\star})$, so all fibers have equal size. Since the codomain has size 2053, each value has exactly $|\mathcal{A}|/2053$ preimages. The final statement follows immediately. □

Proposition 5.3.2 (Conservative Reduced Mode bias bound)**.** *Fix one word polynomial in an* (k, n) *scheme and an adversary holding* $t < k$ *shares. Let* $m = n - t$ *be the number of unseen share positions. Then the Reduced Mode range restriction excludes at most a* $5m/2053$ *fraction of otherwise compatible candidates. Equivalently, the surviving fraction is at least*

$$1 - \frac{5m}{2053} = \frac{2053 - 5m}{2053}.$$

For an ℓ*-word mnemonic, this yields a conservative candidate-space reduction bound of at most*

$$\ell \cdot \log_2 \left(\frac{2053}{2053 - 5m} \right)$$

bits.

Proof sketch. The forbidden set for a Reduced Mode word share is

$$F = \{2048, 2049, 2050, 2051, 2052\},$$

which has size 5.

If $t = k - 1$, then each candidate secret determines a unique polynomial. At any unseen share index x, Lagrange interpolation with nodes $\{0, x_1, \ldots, x_{k-1}\}$ gives

$$f(x) = \alpha_x s + \beta_x$$

for constants $\alpha_x, \beta_x \in \mathrm{GF}(2053)$ determined by the observed shares, with

$$\alpha_x = \prod_{i=1}^{k-1} \frac{x - x_i}{-x_i} \neq 0$$

because $x \notin \{0, x_1, \ldots, x_{k-1}\}$. Hence $s \mapsto f(x)$ is an affine bijection, so at most 5 candidate secrets are rejected at that position. By union bound over the m unseen positions, at most $5m$ candidate secrets are rejected in total.

If $t < k - 1$, then after fixing the candidate secret and the observed shares, the compatible polynomials form an affine space of positive dimension. By Lemma 5.3.1, for any unseen share index x, exactly a $5/2053$ fraction of these compatible polynomials satisfy $f(x) \in F$. Hence the rejection fraction per unseen position is exactly $5/2053$, and by union bound the total rejection fraction is at most $5m/2053$.

No independence assumption is required. □

Concrete 24-word bounds. For the configurations emphasized in this work, the conservative 24-word bounds are:

$$\begin{aligned} \text{2-of-3 with } t = 1\ (m = 2): &\ \leq 0.169 \text{ bits}, \\ \text{2-of-4 with } t = 1\ (m = 3): &\ \leq 0.255 \text{ bits}, \\ \text{3-of-5 with } t = 2\ (m = 3): &\ \leq 0.255 \text{ bits}. \end{aligned}$$

Across all unauthorized sets of these same topologies, the worst case is $m = 5$, giving

$$24 \cdot \log_2\left(\frac{2053}{2028}\right) \approx 0.424$$

bits.

5.4 Substitution Detection

Arithmetic Lock: zero substitution detection. Following the cheater framework of Tompa and Woll [18], an adversary with physical access to one share can choose arbitrary word values and compute consistent row checksums, column checksums, and GIC. The modified share passes all $r + 4$ check equations. After Lagrange interpolation with the fake share and $k-1$ genuine shares, the recovered result is internally consistent (linearity is preserved) but produces a wrong mnemonic. No per-share inspection can distinguish a substituted share from a genuine one.

Pre-recovery detection (manifest-dependent). If an uncompromised manifest is available, certain checks can be performed *before* recovery on a per-share basis:

1. **Share-bound GIC comparison** (all modes): compare the share's GIC against the manifest entry for that share number. This detects accidental corruption or cross-session share mixing, but *not* deliberate substitution—an adversary can choose word values that produce the same GIC by preserving the word sum modulo 2053.
2. **Audit Hash** (computational mode): recompute SHA-256(Core Payload) from the scanned/imported share and compare against the manifest record. This detects *both* accidental and deliberate substitution, since the adversary cannot produce a different payload with the same hash (SHA-256 collision resistance).

Both checks are conditional: if the adversary also compromises the manifest, pre-recovery detection fails. Manifest content—Session Batch ID, Blinded Identity, share indices, and audit hashes—contains no plaintext secret shares, but it is not information-theoretically neutral: the Blinded Identity is a mnemonic-dependent verification tag. Under the intended threat model, this metadata may still be printed on an untrusted printer because it does not by itself reveal the mnemonic. The audit hash provides a verification oracle for candidate secrets, but the search space remains the full mnemonic keyspace (2^{128} to 2^{256}) because SHA-256 covers all word values jointly and does not enable per-word testing.

Post-recovery detection (unconditional). After Lagrange interpolation produces the recovered mnemonic, substitution is detected by:

1. **BIP39 checksum** ($\sim$99.6% for 24 words): independent non-linear check on the recovered mnemonic.
2. **Identity Lock** (computational mode): Blinded Identity mismatch with $\sim 2^{96}$ (Full) or $\sim 2^{64}$ (Reduced) confidence.
3. **RVA** (all modes): target-wallet address mismatch ($\sim 2^{60}$ for bech32/bech32m).

These checks require only the recovered mnemonic and do not depend on a manifest.

In manual mode without RVA, post-recovery substitution detection relies solely on BIP39 checksum and operational mitigations (tamper-evident storage, distributed manifests).

5.5 Manual and Computational Trade-offs

Both modes execute identical mathematics over GF(2053); they differ only in the executor (human vs. silicon). Mode selection affects operational security, not cryptographic properties. Shares generated in one mode are fully usable in the other.

Computational mode provides sub-5-minute operation for both sharing and recovery, with Transport Lock, Identity Lock, and RVA. Standard implementation risks apply (side-channels, memory safety, compromised dependencies or firmware, and supply chain). Recommended mitigations include air-gapped execution, constant-time comparisons where applicable, explicit memory cleanup, minimal dependencies, a narrow trusted I/O path using non-networked peripherals for secret-bearing material, and open-source code for independent audit.

Manual mode enables the complete lifecycle—both sharing and recovery—when devices are unavailable, obsolete, or untrusted. Manual sharing requires 2–5 hours; manual recovery requires 30–60 minutes (Section 7). Unique risks include extended physical exposure, impression attacks on paper, persistence of intermediate worksheets, and arithmetic errors (mitigated by checksum architecture with $d = 4$).

6 Worked Example in GF(13)

We illustrate the full sharing and recovery lifecycle using GF(13) for tractable arithmetic. The real scheme uses GF(2053); the structure is identical. We use the same column tags $\{10, 20, 30\}$ and totals $T_C = 60$ as the real scheme, deferring modular reduction to the evaluation step. This example uses a single row ($r = 1$); with multiple rows, each column checksum sums across all rows in that column.

Setup. Toy wordlist (indices 0–12). Mnemonic: $w_1 = 3$, $w_2 = 5$, $w_3 = 7$ (one row, three columns). Row tag $\tau_1^R = 1$, column tags $\tau_1^C = 10$, $\tau_2^C = 20$, $\tau_3^C = 30$. Row total $T_R = 1$, column total $T_C = 60$. Scheme: 2-of-3.

6.1 Sharing

Step 1: Word polynomials. Sample $a_1 \in \{1, \ldots, 12\}$ for each word:

$$f_{w_1}(x) = 3 + 4x \pmod{13}$$
$$f_{w_2}(x) = 5 + 6x \pmod{13}$$
$$f_{w_3}(x) = 7 + 1x \pmod{13}$$

Evaluate at $x \in \{1, 2, 3\}$ to produce word shares:

x	w_1	w_2	w_3
1	7	11	8
2	11	4	9
3	2	10	10

Step 2: Row checksum and incremental validation.

$$f_{R_1}(x) = (3{+}5{+}7{+}1) + (4{+}6{+}1)x = 3 + 11x \pmod{13}$$

Evaluate and validate immediately on each share:

x	w_1	w_2	w_3	R_1	Check: $(w_1{+}w_2{+}w_3{+}1)$ mod 13
1	7	11	8	1	(7+11+8+1) = 27 mod 13 = 1 ✓
2	11	4	9	12	(11+4+9+1) = 25 mod 13 = 12 ✓
3	2	10	10	10	(2+10+10+1) = 23 mod 13 = 10 ✓

Step 3: Column checksums.

$$\begin{aligned} f_{C_1}(x) &= (3{+}10) + 4x = 13 + 4x \pmod{13} \\ f_{C_2}(x) &= (5{+}20) + 6x = 25 + 6x \pmod{13} \\ f_{C_3}(x) &= (7{+}30) + 1x = 37 + 1x \pmod{13} \end{aligned}$$

Evaluate and validate on each share (showing share 1): (7+10) mod 13 = 4 = C_1 ✓; (11+20) mod 13 = 5 = C_2 ✓; (8+30) mod 13 = 12 = C_3 ✓.

Step 4: GIC. Base: $f_{\text{GIC}}(x) = (3{+}5{+}7{+}1{+}60) + (4{+}6{+}1)x = 76 + 11x \pmod{13}$. Share-bound: $\text{GIC}(x) = (76 + 11x + x) \bmod 13 = (76 + 12x) \bmod 13$.

Three equivalent validation paths (share 1, GIC = 10):

$$\begin{aligned} \text{Words: } &(7{+}11{+}8{+}1{+}60{+}1) \bmod 13 = 88 \bmod 13 = 10 \checkmark \\ \text{Rows: } &(1{+}60{+}1) \bmod 13 = 62 \bmod 13 = 10 \checkmark \\ \text{Cols: } &(4{+}5{+}12{+}1{+}1) \bmod 13 = 23 \bmod 13 = 10 \checkmark \end{aligned}$$

Step 5: Share assembly. Final share table:

x	w_1	w_2	w_3	R_1	C_1	C_2	C_3	GIC
1	7	11	8	1	4	5	12	10
2	11	4	9	12	8	11	0	9
3	2	10	10	10	12	4	1	8

6.2 Recovery from shares 1 and 3

Step 1: Per-share GIC validation. Share 1: (7+11+8+1+60+1) mod 13 = 88 mod 13 = 10 = GIC ✓. Share 3: (2+10+10+1+60+3) mod 13 = 86 mod 13 = 8 = GIC ✓.

Step 2: Lagrange coefficients and sanity check. $\gamma_1 = 3/(3{-}1) = 3 \cdot 7 = 8 \pmod{13}$, $\gamma_3 = 1/(1{-}3) = 1 \cdot (-2)^{-1} = 6 \pmod{13}$.

Sanity: $(8 \cdot 1 + 6 \cdot 3) \bmod 13 = 26 \bmod 13 = 0$ ✓.

Step 3: Row-by-row interpolation with incremental validation. Interpolate row 1 words and row checksum, then validate before proceeding:

$$\begin{aligned} \hat{w}_1 &= (8 \cdot 7 + 6 \cdot 2) \bmod 13 = 68 \bmod 13 = 3 \\ \hat{w}_2 &= (8 \cdot 11 + 6 \cdot 10) \bmod 13 = 148 \bmod 13 = 5 \\ \hat{w}_3 &= (8 \cdot 8 + 6 \cdot 10) \bmod 13 = 124 \bmod 13 = 7 \\ \hat{R}_1 &= (8 \cdot 1 + 6 \cdot 10) \bmod 13 = 68 \bmod 13 = 3 \end{aligned}$$

Incremental check: $(\hat{w}_1 + \hat{w}_2 + \hat{w}_3 + 1) \bmod 13 = (3{+}5{+}7{+}1) \bmod 13 = 3 = \hat{R}_1$ ✓. Row passes; proceed.

Step 4: Global validation. Interpolate column checksums and GIC: $\hat{C}_1 = (8 \cdot 4 + 6 \cdot 12) \bmod 13 = 104 \bmod 13 = 0$; $\hat{C}_2 = (8 \cdot 5 + 6 \cdot 4) \bmod 13 = 64 \bmod 13 = 12$; $\hat{C}_3 = (8 \cdot 12 + 6 \cdot 1) \bmod 13 = 102 \bmod 13 = 11$.

Column checks: (3+10) mod 13 = 0 = $\hat{C}_1$ ✓; (5+20) mod 13 = 12 = $\hat{C}_2$ ✓; (7+30) mod 13 = 11 = $\hat{C}_3$ ✓.

Interpolate GIC: $\hat{\text{GIC}} = (8 \cdot 10 + 6 \cdot 8) \bmod 13 = 128 \bmod 13 = 11$. The $+x$ terms cancel automatically.

Three validation paths:

$$\text{Words: } (3{+}5{+}7{+}1{+}60) \bmod 13 = 76 \bmod 13 = 11 = \hat{\text{GIC}} \checkmark$$
$$\text{Rows: } (3{+}60) \bmod 13 = 63 \bmod 13 = 11 = \hat{\text{GIC}} \checkmark$$
$$\text{Cols: } (0{+}12{+}11{+}1) \bmod 13 = 24 \bmod 13 = 11 = \hat{\text{GIC}} \checkmark$$

Step 5: Output. Recovered mnemonic: $(w_1, w_2, w_3) = (3, 5, 7)$. Convert to wordlist entries and validate the checksum (in the real scheme, BIP39 validation).

7 Implementation Notes and Preliminary Evidence

Reference implementations. Multiple MIT-licensed implementations enable independent verification:

- JavaScript/TypeScript: npm package `@grifortis/schiavinato-sharing`
- Python: library implementation
- HTML: offline single-file tool for air-gapped environments

All are experimental, unaudited, and **must not** be used for significant holdings. Repositories: https://github.com/GRIFORTIS/schiavinato-sharing.

Test vectors. Version-scoped test vectors in `test_vectors/` enable cross-implementation compatibility validation, including polynomial coefficients, share values, and expected checksums.

Author-measured manual timing benchmarks (v0.5.0). Measured under the current protocol (position-bound row checksums, column checksums, GIC with row/column totals):

Configuration	Manual sharing	Manual recovery
12-word, 2-of-3	∼2 hours	∼30 minutes
24-word, 3-of-5	∼5 hours	∼60 minutes

Manual sharing includes entropy generation, polynomial evaluation, all checksum computations, share transcription, and final validation. Manual recovery includes per-share GIC validation, Lagrange interpolation (using pre-computed coefficient tables), row/column/GIC cross-checks, and BIP39 word conversion.

Manual entropy generation. The Schiavinato RNG uses a $7 \times 7 \times 7 \times 6 = 2058$ rejection sampling scheme. A tuple (a, b, c, d) with $a, b, c \in \{0, \ldots, 6\}$, $d \in \{0, \ldots, 5\}$ maps to $t = (((a \cdot 7 + b) \cdot 7 + c) \cdot 6 + d) \in \{0, \ldots, 2057\}$. Reject $t \geq 2053$ (probability $5/2058 \approx 0.24\%$). For the highest coefficient, additionally reject $t = 0$. In timed runs, coefficient generation averaged ∼45 seconds each, yielding ∼36 minutes for the 48 coefficients required by a 24-word, 3-of-5 configuration.

Usability context. An informal pilot with four non-technical participants (ages 28–72) on an early prototype suggested that (i) procedural retention was poor after 13 months, indicating that self-documenting materials and checkpointed workflows are important for inheritance scenarios, and (ii) manual Lagrange computation was the primary bottleneck, motivating the pre-computed coefficient tables in the current protocol. This pilot pre-dated the current checksum architecture and tested a harder configuration (4-of-16); it is not representative of v0.5.0 performance.

8 Discussion

8.1 Limitations

- **No manual Identity Lock without computation:** purely manual-arithmetic functions over GF(2053) are low-degree polynomials providing at most ∼11 bits of discrimination per constraint—insufficient for meaningful identity binding. The RVA addresses this but requires a one-time computational step during the sharing ceremony.
- **Arithmetic Lock is error-detection, not authentication:** the checksum architecture detects random errors ($d = 4$) but cannot resist intentional substitution by an adversary with physical access.
- **No professional audit:** reference implementations have not undergone independent security audit or formal verification.
- **Usability evidence is preliminary:** timing benchmarks are from the protocol author; no controlled third-party usability study exists for v0.5.0.

8.2 Falsifiability Criteria

F1 (Confidentiality): Formal analysis demonstrates coupled constraints enable $t < k$ shares to narrow search below 2^{256}.

F2 (Manual feasibility): Controlled study ($n \geq 20$) shows $< 50\%$ success rate for manual recovery.

F3 (BIP39 compatibility): Recovered mnemonics fail validation or produce incorrect addresses in major implementations.

F4 (Detection distance): A 3-cell error pattern is found that passes all $r + 4$ check equations.

F5 (Implementation): Security audit identifies a vulnerability with attack complexity $< 2^{128}$.

8.3 Path Toward Production Readiness

Phase 1: Seek cryptographic peer review; independently reproduce test vectors.

Phase 2: Gather third-party usability evidence for the v0.5.0 manual workflow (success rates, time distributions, common failure modes).

Phase 3: Pursue independent code review and/or professional security audit as appropriate to deployment context.

Experimental status maintained until all phases complete.

9 Conclusion

Schiavinato Sharing demonstrates that threshold security, native BIP39 compatibility, and manual/computational complementary operation can be combined using prime-field GF(2053) arithmetic with a linear integrity architecture. The checksum system has minimum detection distance $d = 4$ with undetected-error probability $1/2053^{r+4}$ (where $r = \ell/3$), providing strong assurance against random arithmetic and transcription errors. The mnemonic-bound Identity Lock and wallet-bound Recovery Verification Address provide post-recovery substitution detection for computational and manual modes respectively. Pre-recovery, manifest Audit Hashes (computational mode) detect deliberate substitution, while manifest GIC comparison detects accidental corruption or cross-session mixing in all modes. The Arithmetic Lock alone cannot detect intentional tampering.

Information-theoretic confidentiality for $t < k$ applies to the arithmetic share layer: the GF(2053) word shares together with the row checksums, column checksums, and GIC. Optional authenticity metadata, particularly the Blinded Identity and Recovery Verification Address, are outside that unconditional-secrecy claim: they provide verification tags that strengthen

substitution detection rather than preserve perfect secrecy. Reduced Mode introduces a small, conservatively bounded range-restriction bias whose exact size depends on the mnemonic length and the number of unseen share positions. For 24-word examples, the bound remains below 0.26 bits in the common maximal-unauthorized cases and below 0.43 bits across all unauthorized sets of the currently emphasized topologies. The construction supports recursive composition for complex governance structures while maintaining manual verifiability.

This document is presented for peer review rather than as a production recommendation. We invite critical analysis, independent replication, and falsification along the criteria stated in Section 8.

Acknowledgments

The author acknowledges Adi Shamir's foundational 1979 secret sharing scheme [14], the BIP39 standard [3], and analysis of existing schemes (SLIP39, SSKR, Codex32, SeedXOR) that informed this design. The author thanks Jean Martina, whose feedback on an earlier version prompted the Recovery Verification Address design. This work was developed independently by GRIFORTIS without external funding. All errors remain the author's responsibility.

References

[1] A. Beimel, "Secret-Sharing Schemes: A Survey," *Coding and Cryptology, IWCC 2011*, Springer LNCS, vol. 6639, pp. 11–46, 2011.

[2] P. Wuille, "BIP-32: Hierarchical Deterministic Wallets," *Bitcoin Improvement Proposals*, 2012. [Online]. Available: https://bips.dev/32/.

[3] M. Palatinus, P. Rusnak, A. Voisine, S. Bowe, "BIP-39: Mnemonic code for generating deterministic keys," *Bitcoin Improvement Proposals*, 2013. [Online]. Available: https://bips.dev/39/.

[4] M. Palatinus and P. Rusnak, "BIP-44: Multi-Account Hierarchy for Deterministic Wallets," *Bitcoin Improvement Proposals*, 2014. [Online]. Available: https://bips.dev/44/.

[5] G. R. Blundo, A. De Santis, D. R. Stinson, and U. Vaccaro, "Graph decompositions and secret sharing schemes," *EUROCRYPT '92*, Springer LNCS, vol. 658, pp. 1–24, 1993.

[6] V. Buterin, "Why we need wide adoption of social recovery wallets," Blog post, Jan. 2021. https://vitalik.ca/general/2021/01/11/recovery.html.

[7] A. Poelstra, L. O'Connor, S. Corallo, "BIP-93: codex32: Checksummed SSSS-aware BIP32 seeds," *Bitcoin Improvement Proposals*, Draft, 2023. [Online]. Available: https://bips.dev/93/.

[8] R. Cramer, I. Damgård, and U. Maurer, "General secure multi-party computation from any linear secret-sharing scheme," *EUROCRYPT 2000*, Springer LNCS, vol. 1807, pp. 316–334, 2000.

[9] S. Eskandari, D. Barrera, E. Stobert, and J. Clark, "A First Look at the Usability of Bitcoin Key Management," *Proc. USEC 2015 (Workshop on Usable Security)*, 2015. doi: https://doi.org/10.14722/usec.2015.23015.

[10] National Institute of Standards and Technology (NIST), *FIPS PUB 180-4: Secure Hash Standard (SHS)*, Aug. 2015. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf.

[11] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Feb. 1997. https://www.rfc-editor.org/rfc/rfc2104.

[12] R. Schiavinato Lopez, *Schiavinato Sharing v0.4.1*, GRIFORTIS, 2025. Available at https://github.com/GRIFORTIS/schiavinato-sharing.

[13] Coinkite, "SeedXOR: XOR-based secret sharing for BIP39 mnemonics," 2021. [Online]. Available: https://seedxor.com/.

[14] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.

[15] P. Rusnak, A. Kozlik, O. Vejpustek, T. Susanka, M. Palatinus, and J. Hoenicke, "SLIP-0039: Shamir's Secret-Sharing for Mnemonic Codes," SatoshiLabs Improvement Proposals, Final, 2019. [Online]. Available: https://github.com/satoshilabs/slips/blob/master/slip-0039.md.

[16] W. McNally and C. Allen, "UR Type Definition for Sharded Secret Key Reconstruction (SSKR)," Blockchain Commons Research, BCR-2020-011, ver. 1.0.1, 2020 (rev. 2021). [Online]. Available: https://github.com/BlockchainCommons/Research/blob/master/papers/bcr-2020-011-sskr.md.

[17] D. R. Stinson, *Cryptography: Theory and Practice*, 3rd ed. Chapman and Hall/CRC, 2006.

[18] M. Tompa and H. Woll, "How to share a secret with cheaters," *Journal of Cryptology*, vol. 1, no. 2, pp. 133–138, 1988.

A Digital Payload Encoding (Reference)

This appendix provides the reference byte layout for the digital payload. The canonical specification is maintained at https://github.com/GRIFORTIS/schiavinato-sharing.

A.1 Full Mode Core Payload (62–80 bytes)

Offset	Length	Field
0	1	Protocol version (0x01)
1	1	Flags (word count, nesting, derivation hint)
2–3	2	Threshold k (layer-dependent packing)
4–5	2	Share index x (layer-dependent packing)
6–13	8	Session Batch ID
14–25	12	Blinded Identity
26–63	up to 38	Share data (words + GIC, 12-bit packed)
64–79	16	Transport Hash (SHA-256 truncated)

QR prefix: ASCII `SCHI` (4 bytes). QR version: V5 (37×37), error correction level M.

A.2 Reduced Mode Core Payload (34–51 bytes)

Offset	Length	Field
0	1	Protocol version (0x01)
1	1	Flags (word count, nesting, derivation hint)
2	1	Threshold k
3	1	Share index x
4–7	4	Session Batch ID
8–15	8	Blinded Identity
16–50	up to 35	Share data (words 11-bit + GIC 12-bit, packed)

No Transport Hash (transport integrity via QR error correction + GIC). QR prefix: ASCII `SC` (2 bytes). QR version: V3 (29×29), error correction level M (12/15 words) or L (18/21/24 words).

Row and column checksums are computed from the share data for human-readable display but are not serialized in the digital payload.

B Pre-Computed Lagrange Coefficients

Lagrange coefficients are determined entirely by share indices and contain zero secret information. They may be computed on any device (even untrusted) or verified against multiple sources.

Table 5: Lagrange coefficients in GF(2053)

Scheme	**Shares**	**Coefficients** (γ)
2-of-3	$\{1,2\}$	$(2, 2052)$
	$\{1,3\}$	$(1028, 1026)$
	$\{2,3\}$	$(3, 2051)$
2-of-4	$\{1,2\}$	$(2, 2052)$
	$\{1,3\}$	$(1028, 1026)$
	$\{1,4\}$	$(1370, 684)$
	$\{2,3\}$	$(3, 2051)$
	$\{2,4\}$	$(2, 2052)$
	$\{3,4\}$	$(4, 2050)$
3-of-5	$\{1,2,3\}$	$(3, 2050, 1)$
	$\{1,2,4\}$	$(687, 2051, 1369)$
	$\{1,2,5\}$	$(1029, 1367, 1711)$
	$\{1,3,4\}$	$(2, 2051, 1)$
	$\{1,3,5\}$	$(1285, 512, 257)$
	$\{1,4,5\}$	$(686, 1367, 1)$
	$\{2,3,4\}$	$(6, 2045, 3)$
	$\{2,3,5\}$	$(5, 2048, 1)$
	$\{2,4,5\}$	$(1372, 2048, 687)$
	$\{3,4,5\}$	$(10, 2038, 6)$